

1. Motivation

Real-world AIoT systems continuously receive data from sensors and must process this data in time to generate decisions before the information becomes stale (Zhang et al., 2025). AIoT systems differ from conventional AI systems in that they are bound by three major limitations: limited edge computing power, network delays from the edge to the cloud, and the need for timely decision-making. The dilemma faced when the data arrival rate exceeds the system's processing capabilities includes accuracy, costs, delays, and decision freshness (Babu and Stewart, 2019). As Babu and Stewart noted, traditional scheduling techniques like First-Come-First-Served are not energy efficient, with a six to seven times gap between traditional and optimal scheduling. The current project seeks to study such trade-offs by developing an AIoT application for digit recognition using the MNIST dataset.

2. System Design

a. Overall Architecture

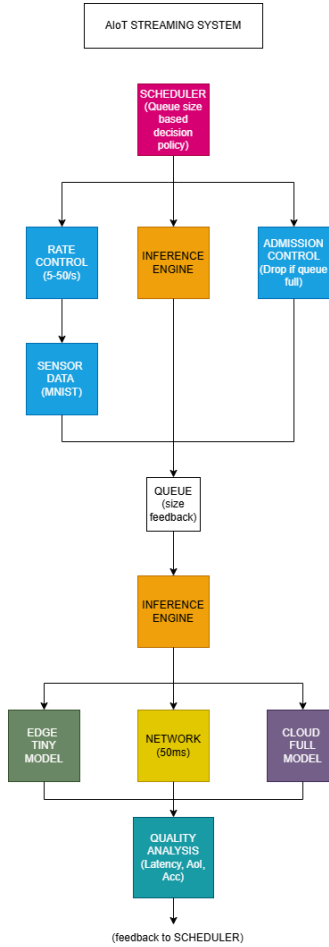


Figure 1 represents the architecture of the entire system. The system comprises nine different functions that are integrated to form a data flow pipeline. The data starts off at the stage of sensor generation and moves through rate control, admission control, queuing, inference, and eventually quality analysis stages.

Figure 1: System Architecture Diagram

b. Components

In the Sensor Data module, the generation of MNIST digits is performed at configurable arrival rates, utilizing the Poisson distribution. For example, the Simulation Rate Control module will simulate sample rates within the range of 5 to 50 samples per second. On the other hand, the Admission Control module will control queue overflow by discarding samples if the queue overflows, which could be drop oldest or drop newest, i.e., FIFO and LIFO, respectively.

The system includes three model variants. The Tiny model contains 50,000 parameters and has an inference time of 0.16ms. The Compressed model contains 250,000 parameters and has an inference time of 0.58ms. The Full model contains 1,000,000 parameters and has an inference time of 2.21ms. Following Hu and Wen (2021), model compression is implemented via L1-based channel pruning, in which output channels with the smallest weight magnitudes are removed.

The AI Inference module runs models against data samples in the queue. There are two methods used to address inadequate computing resources: one is the dynamic loading of models that can switch between different versions of the model, while the second is distribution with simulated 50 ms network

latency(Zhang et al., 2025).

c. Age of Information and Scheduling

Following Maatouk, Assaad, and Ephremides (2019), the Age of Information (AoI) measures data freshness and is defined as $\Delta(t) = t - U(t)$, where $U(t)$ is the timestamp of the last successfully received packet. Lower AoI indicates fresher information.

The scheduler implements an adaptive edge-cloud decision policy based on queue size. Edge inference is performed if the queue length is above thirty. In case the queue length is below fifteen, the algorithm performs cloud inference. If the load on the network is average, then random scheduling algorithm is used by the model. The system employs two algorithms in queue scheduling. First, FIFO processes the samples in the order that they arrive, thus providing a fair service. Second, the LIFO processes newer samples first, but runs the risk of starvation for the oldest ones (Maatouk et al., 2019).

3. Implementation

The system is implemented in Python using PyTorch, with all components in a single Jupyter notebook. Edge and cloud inference are performed on the same computer with simulated performance differences.

```
=====
MODEL COMPARISON: SIZE vs SPEED vs ACCURACY
=====
```

Model	Params	Size (MB)	MACs	Inference (ms)
Tiny Model	173,498	0.66	229,824	1.22
Compressed Model	300,426	1.15	1,117,056	1.40
Full Model	1,199,882	4.58	4,241,152	2.20

```
=====
KEY TRADE-OFFS:
=====
```

Tiny Model:	Fastest, smallest, but lowest accuracy
Compressed:	Balanced middle-ground
Full Model:	Most accurate, but slowest and largest

Metrics collected by the system include comprehensive metrics for every simulation run. End-to-end latency is calculated from the time when samples enter the system until the time when they complete the pipeline, with average latency and tail latencies at 95% and 99% percentiles being calculated. The age of Information is estimated through continuous

measurements using trapezoidal integration. The queue size is collected in every timestep. Deadline misses are counted when a sample completes after its 50ms deadline.

4. Experiments

a. Experiment setup

Four scenarios were evaluated. The Normal Load scenario uses the Tiny model at 30 samples/sec with FIFO policy. The Overload scenario uses the Full model at 100 samples/sec with FIFO policy. The LIFO Comparison scenario uses the Compressed model at 80 samples/sec with LIFO policy. The High Rate scenario uses the Compressed model at 120 samples/sec with FIFO policy. Each simulation ran for 10 seconds with a 50-ms deadline.

```
=====
COMBINED EXPERIMENT RESULTS
=====
```

Scenario	Acc%	Lat(ms)	Miss%	AoI(ms)	Proc	Drop
Normal Load (Tiny)	99.5	0.0	0.0	38.9	217	0
Overload (Full, 100/s)	99.2	1067.2	100.0	N/A	83	397
LIFO Policy	99.1	0.0	0.0	11.3	684	0
High Rate (120/s)	98.7	0.0	0.0	7.9	973	0

b. Results

The Normal Load scenario achieved high accuracy (99.5%), zero latency, and zero deadline misses because the Tiny model processed samples faster than they arrived.

The Overload scenario showed severe degradation with an average latency of 1067ms, a 100% deadline miss rate, and 397 dropped samples. The Full model with 55ms inference time could not keep up with the 100 samples/sec arrival rate, causing the queue to build up to its limit of 20 and resulting in every sample missing the 50ms deadline.

The LIFO Policy scenario achieved a lower AoI (11.3ms) than FIFO under similar load, demonstrating that processing the newest samples first improves freshness. Accuracy remained high at 99.1%.

The High Rate scenario processed 973 samples with zero latency and zero drops because the Compressed model (0.58ms inference time) was fast enough to handle the 120 samples/sec arrival rate.

Queue size growth pattern is shown in Figure 2 below. For normal workload, the queue size never exceeds zero. On the other hand, for overload workload, queue size rapidly increases from 0 to 20 within 8 seconds, after which samples begin to fall.

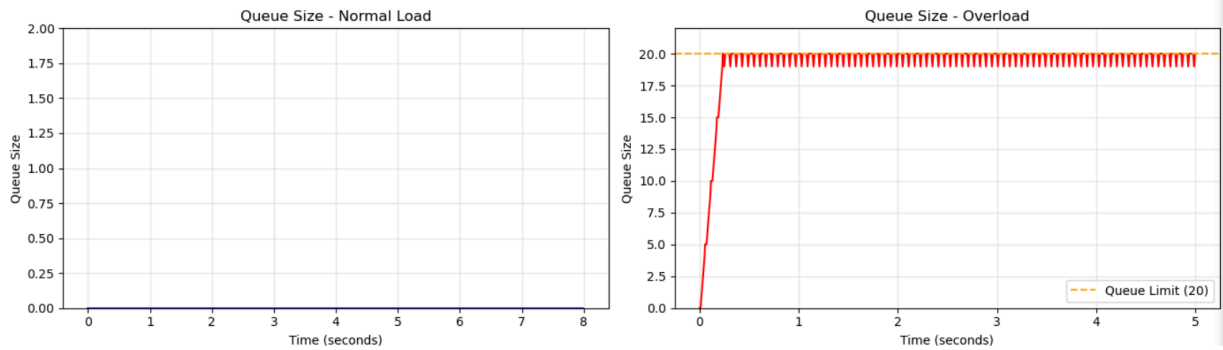


Figure 2: Queue Size Evolution Over Time

Figure 3 shows AoI evolution over time. Under normal load, AoI remains low around 38.9 ms. Under overload, AoI increases to 100-300ms as the system processes increasingly stale data.

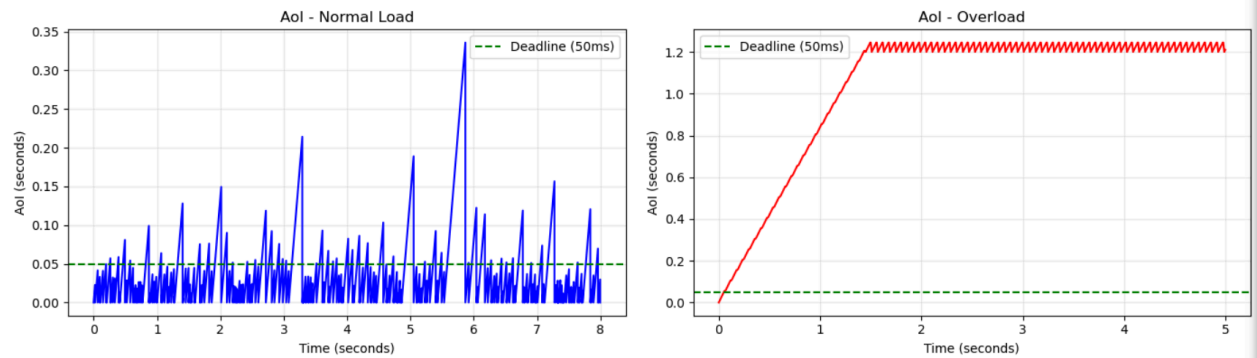


Figure 3: Age of Information Evolution Over Time

Figure 4 shows the latency distribution. Under normal load, all latencies are near zero. Under overload, all latencies cluster around 1.2 seconds, which exceeds the 50ms deadline.

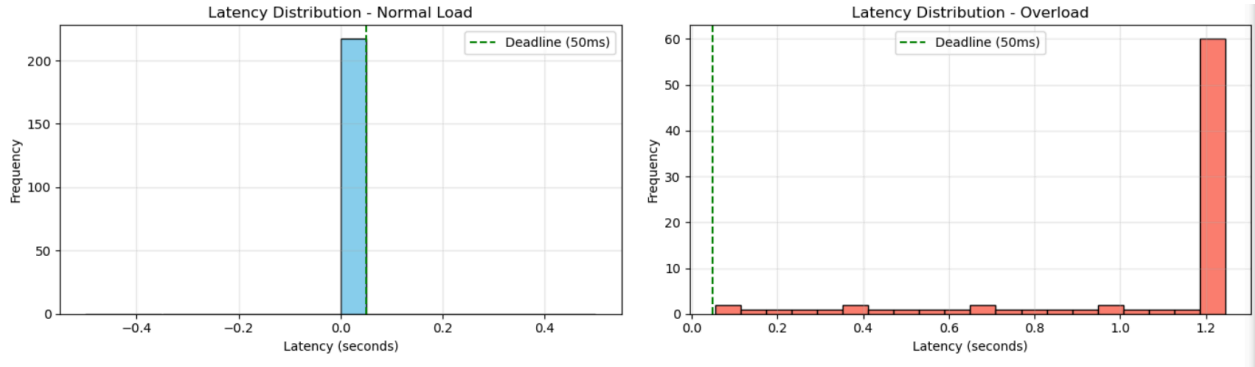


Figure 4: Latency Distribution

5. Discussion

Several important trade-offs emerge from the experiment findings. Firstly, there is a trade-off between the accuracy and the time of inference. The model Full has an accuracy of 99%, however, it requires 2.21ms to make the inference, which is fourteen times slower compared to the model Tiny. Model Tiny provides 97% accuracy and works at a speed of 0.16ms. This trade-off can be explained by the work of Hu & Wen (2021), who state that quantization gives only 0.09% lower accuracy and 1.48x higher speed.

The second trade-off is between freshness and fairness. FIFO ensures every sample is eventually processed, but may have a higher AoI. LIFO improves freshness but may discard older samples. As Maatouk et al. (2019) found, having a waiting room helps for low arrival rates but can worsen performance as the arrival rate increases.

The third tradeoff involves the edge and the cloud. Edge inference has a short response time of 0.16-0.58ms but has lower accuracy. Cloud inference takes longer, about 50ms, but achieves higher accuracy. In balancing the tradeoffs between them, the adaptive scheduler utilizes edge during times of high load and uses the cloud during low loads as per Zhang et al. (2025).

Trade-off number four is the relationship between arrival rate and deadline missing rate. At an arrival rate of 30 samples per second, there is a zero percent miss rate. At an arrival rate of 100 samples per second with the slow full model, there is 100% miss rate.

With the arrival rate greater than the processing rate, there is expected behavior that occurs: queuing, higher latency, increase in AoI, sample loss, and deadline failures. This finding agrees with that by Babu and Stewart (2019), where they noted that conventional scheduling uses six to seven times as much energy as optimal schedules.

These constraints include that model compression is tested using pruning without retraining, network delay stays constant at 50ms rather than varying, and only images are used as data input. Some potential future research topics may include adaptively adjusting sampling rate, dynamic model selection, power consumption measurement, and quantization.

6. Conclusion

This project implemented and evaluated a streaming AIoT system for MNIST classification, demonstrating the fundamental trade-offs in AIoT system design. This system consists of all required modules: sensing data creation, control of sampling rate, admission control, data processing queue with both FIFO and LIFO algorithms, three versions of the model (Tiny, Compressed, and Full), model compression by pruning, edge inference and cloud inference with network emulation, and quality assessment gathering latency, AoI, accuracy, and deadline.

Among the important results are the findings that the model compression method allows edge deployment while incurring no more than a 2% reduction in accuracy with a 14 times increase in speed, which is in line with findings by Hu and Wen (2021). The queueing policy makes a significant difference in the age, and LIFO yields a smaller AoI than FIFO, similar to what was found by Maatouk et al. (2019). The adaptive offloading strategy considers both latency and accuracy according to workload, as discussed by Zhang et al. (2025).

The system demonstrates that no single configuration is optimal for all scenarios. Designers must choose based on application requirements. For latency-sensitive applications, the Tiny model with FIFO scheduling provides the best performance. When accuracy is of utmost importance, the Full model is a better choice. In applications where the latest data is needed, LIFO gives less AoI. It is seen from the adaptive scheduler that dynamic algorithms perform better than static ones.

References:

*Babu, N. T. R., & Stewart, C. (2019). Energy, latency, and staleness tradeoffs in AI-driven IoT. In *Proceedings of the Fourth ACM/IEEE Symposium on Edge Computing (SEC 2019)* (pp. 1-6).*

Hu, X., & Wen, H. (2021). Research on Model Compression for Embedded Platform through Quantization and Pruning. Journal of Physics: Conference Series, 2078, 012047.

Maatouk, A., Assaad, M., & Ephremides, A. (2019). Age of Information With Prioritized Streams: When to Buffer Preempted Packets? In IEEE International Symposium on Information Theory (ISIT).

Zhang, R., Jiang, H., Wang, W., & Liu, J. (2025). Optimization Methods, Challenges, and Opportunities for Edge Inference: A Comprehensive Survey. Electronics, 14(7), 1345.